```
                return ("반례", f"i: 대상 자신을 얼린 선형 문제(μ = {mu:.3g}, 정확한 흐름은 유계)에 돌리면 t = {tt:.4g}에서 출력이 무한대로 간다(반례: 대상이 정확한 흐름에 없는 증폭을 만든다)")
            fl_ = math.exp(min(mu * tt, 700)) * n0 * (1 + EPS_AMP) * (1 + 4 * U)
            r = float(np.linalg.norm(W * yy)) * (1 - g) / fl_
            if r > 1:
                return ("반례", f"i: 대상 자신을 얼린 선형 문제(로그노름 μ = {mu:.3g})에 돌리면 t = {tt:.4g}에서 출력의 노름이 정확한 흐름의 한계의 {r:.3g}배이다(반례: 대상이 정확한 흐름에 없는 증폭을 만든다)")
            if r > worst[0]: worst = (r, tt)
    return ("증명", f"i: 목록 밖 방법이라 대상 자신을 얼린 선형 문제(μ = {mu:.3g})에 오차 방향·최대 증폭 방향·임의 방향의 세 시작값으로 돌렸다. "
            f"t = {t_end:.4g}까지 출력은 정확한 흐름의 한계 안이다(최대 비 {worst[0]:.6g} ≤ 1, 증명). 이 세 시작값에서 에러는 증폭 때문이 아니다; "
            f"모든 시작값에 대한 주장은 목록 밖 방법에는 하지 않는다(정의, 15절)")


class _LinSys:
    """minimal stand-in for Sys on the frozen linear problem (F = J y, no reaction): used by the rounding bounds"""
    def __init__(s, J, W):
        s.m, s.N = 1, J.shape[0]; s.L = J; s.D = [1.0]; s.src = np.zeros(J.shape[0]); s.Jnz = []; s.sw = W
        s.spec = dict(R=lambda x: [0]); s._kr = 4
        s.Rabs_gen = lambda *u: [np.zeros(J.shape[0])]
```

six_w.py — 육하원칙 보고의 글 출력과 경로 A·B 증명서의 보고

```
"""six_w: the six-question report (육하원칙 보고) — formatting and the path-A / path-B adapters.
  format_report(rep)                  the report as Korean text, one line per question with its status
  nonlinear_report(best, S, x)        the six questions from a path-A certificate (cert.certify / certify_best)
  pathB_report(res, rows, totals)     the six questions from a path-B judgment (pde_judge(_nd).run + localize)
The verification of a time scheme itself (does it have an error?) is verify.verify; every line it writes is
증명 (proof), 반례 (counterexample), 정의 (definition by an exact decomposition); there is no 보류 (undecided)."""
import math
import numpy as np

U = 2.0 ** -53
Q6 = ("판정", "누가", "무엇을", "언제", "어디서", "왜", "어떻게")

def nonlinear_report(best, S=None, x=None, name="대상 스킴"):
    """six questions from a path-A certificate: best = certify(...)[1] or certify_best(...)[1]; x: node positions (optional)"""
    rep = {}; proved = bool(best.get("proved_full")); tstar = best.get("t_star", 0.0)
    rep["누가"] = dict(status="정의", text=f"{name} (출력 값만 받아 에르미트로 이은 후보)")
    rep["무엇을"] = dict(status="증명" if proved else "정의", text=(f"[0, T]에서 오차 ≤ {best['eps']:.3e}" if proved else
                         (f"[0, {tstar:.4g}]까지 증명, 그 뒤는 증명되지 않음" if tstar > 0 else "증명된 구간이 없음(첫 체크포인트 구간부터 닫히지 않음)")))
    rep["언제"] = dict(status="증명", text=((f"증명된 구간 [0, {tstar:.4g}]" if tstar > 0 else "증명된 구간 없음") + ("" if proved else f"; 막히는 구간 t ∈ [{best.get('t_range90', [0, 0])[0]:.4g}, {best.get('t_range90', [0, 0])[1]:.4g}]")))
    wh = best.get("where") if isinstance(best.get("where"), list) else []
    def loc(w):
        node = w["node"]; pos = f"x = {x[node]}" if x is not None and node < len(x) else f"마디 {node}"
        comp = "모든 성분" if w["component"] == "all" else f"성분 {w['component']}"
        return f"t ∈ [{w['t'][0]:.4g}, {w['t'][1]:.4g}], {comp}, {pos} (몫 {w['share']:.2f})"
    rep["어디서"] = dict(status="정의", text="; ".join(loc(w) for w in wh[:3]) if wh else "국소화 없음")
    names = dict(defect="결함(후보가 방정식을 못 따름)", freezing="동결 오차(야코비안 변화가 빠름)", quadratic="2차 나머지(비선형성이 큼)", jump="이음매(걸음 사이 끊김)")
    dom = best.get("dominant") or (wh[0]["term"] if wh else None)
    rep["왜"] = dict(status="정의", text=f"주된 원인은 {names.get(dom, dom)}" if dom else "원인 없음")
    acts = {"local Taylor": "그 걸음들만 테일러 8차로 다시 만든다", "q*4": "그 걸음들의 하위 걸음을 4배로 늘린다",
            "cap/10": "사전 수준 상한을 낮춘다", "stop": "기법을 바꾼다(10.2절)", None: "필요 없음(증명됨)"}
    rep["어떻게"] = dict(status="처방", text=acts.get(best.get("action"), str(best.get("action"))))
    return rep


def pathB_report(res, rows, totals, name="대상 스킴"):
    """six questions from a path-B judgment: res = Judge(ND).run(...), (rows, totals) = localize(res)"""
    rep = {}
    rep["누가"] = dict(status="정의", text=f"{name} (원래 편미분방정식과 비교, 이산화 순서 무관)")
    rep["무엇을"] = dict(status="증명", text=f"끝 시각에서 L¹ 오차 ≤ {res['bound']:.3e} (가정 H 아래)")
    ser = res.get("series", [])
```

```
    rep["언제"] = dict(status="증명", text=f"모든 조각 끝에서 한도가 성립(조각 {res['pieces']}개)" + (f"; 한도가 가장 빠르게 느는 조각은 t = {max(range(1, len(ser)), key=lambda i: ser[i][1] - ser[i - 1][1], default=0) and ser[max(range(1, len(ser)), key=lambda i: ser[i][1] - ser[i - 1][1])][0]:.4g}" if len(ser) > 1 else ""))
    cause_k = dict(initial="초기 오차", time_scheme="시간 스킴", space="공간 이산", residual="잔차", jump="조각 사이 도약", rounding="반올림")
    def loc(r):
        xs = "조각 경계 전체" if r["x"] is None else " × ".join(f"[{a:.3g}, {b:.3g}]" for a, b in (r["x"] if isinstance(r["x"][0], list) else [r["x"]]))
        tt = f"t = {r['t'][0]:.4g}" if r["x"] is None else f"t ∈ [{r['t'][0]:.4g}, {r['t'][1]:.4g}]"
        return f"{tt}, {xs}, 성분 {r['species']}, {cause_k.get(r['cause'], r['cause'])} (몫 {r['share']:.3f})"
    rep["어디서"] = dict(status="정의", text="; ".join(loc(r) for r in rows[:3]))
    cause_k = dict(initial="초기 오차", time_scheme="시간 스킴", space="공간 이산", residual="잔차", jump="조각 사이 도약", rounding="반올림")
    dom = max(totals, key=totals.get)
    rep["왜"] = dict(status="정의(합은 증명)", text="원인별 몫: " + ", ".join(f"{cause_k.get(k, k)} {v:.3f}" for k, v in sorted(totals.items(), key=lambda kv: -kv[1])))
    fix = dict(initial="초기 격자를 세분한다", time_scheme="그 조각의 걸음을 줄이거나 차수를 올린다", space="그 칸 근처를 세분하거나 재배치한다",
               jump="재배치 간격을 늘리거나 보간을 바꾼다", residual="걸음 또는 격자를 줄인다", rounding="허용오차를 반올림 바닥 위로 올린다")
    rep["어떻게"] = dict(status="처방", text=f"가장 큰 원인({cause_k.get(dom, dom)}): {fix.get(dom)}")
    return rep


def format_report(rep, title="육하원칙 보고"):
    lines = [title]
    for q in Q6:
        if q in rep: lines.append(f"  {q:<4} [{rep[q]['status']}] {rep[q]['text']}")
    return "\n".join(lines)


def _horner(c, h):
    y = c[-1].copy()
    for j in range(len(c) - 2, -1, -1): y = c[j] + h * y
    return y


def _plain_level(best, tt):
    """bound of ||sqrt(w) e(t)|| at time tt from a certificate: the level of the block containing tt divided by the
    smallest species scale of that block (the certificate's norm is ||diag(sigma) sqrt(w) e||)"""
    for (t0, t1, D), sg in zip(best["levels"], best["sigma_blocks"]):
        if t0 - 1e-14 <= tt <= t1 + 1e-14: return float(D) / max(min(sg), 1e-300)
    return float("inf")


def _horner_at(c, s):
    y = c[-1].copy()
    for j in range(len(c) - 2, -1, -1): y = c[j] + s * y
    return y
```

mesh_graph.py — 공간·시간 이산을 격자·망 가중 그래프로: 수식 ↔ 그림의 정확한 왕복(4.4절)

```
"""mesh_graph: 공간 이산(FDM·FVM·FEM)과 시간 이산을 격자·망 위의 가중 그래프로 쓰고, 수식 <-> 그림을 정확히 왕복한다.

  공통 수식(그래프 라플라시안 꼴)
      (L u)_i = (1 / V_i) * sum_{j ~ i} T_ij * (u_j - u_i)                          ... (G)
  그래프의 뜻
      마디 i      = 자유도 하나(FDM: 꼭짓점 값, FVM: 칸 평균, FEM: 모자 함수의 계수)
      마디 무게 V_i = 그 자유도가 맡는 부피(FDM: 사다리꼴 무게, FVM: 칸 너비, FEM: 집중 질량)
      변 무게  T_ij = 두 자유도 사이의 결합(FDM: 1/h, FVM: 면 전달률 1/h, FEM: 요소 강성 1/h_e, 2차원 P1: 코탄젠트 무게)
  그래서 세 공간 이산은 모두 (V, T) 한 쌍이고, 검증기의 입력 (L, w)는 L = 식 (G), w = V이다(4절).
  그래프에서 바로 읽히는 구조(명제 4.1): T >= 0 이면 L의 비대각이 음이 아니다(메츨러), 식 (G)는 행합이 0이다,
  V L = -(가중 그래프 라플라시안)은 대칭이다.

  수식 -> 그림: graph_from_matrix(L, V)가 T_ij = V_i L_ij를 읽어 그래프를 만든다(대칭이 아니면 거부).
  그림 -> 수식: matrix_from_graph(G)가 식 (G)로 행렬을 조립하고, stencil_formula(G, i)가 i행의 식을 기호로 쓴다.
  모든 계수는 sympy 유리수이므로 왕복 비교는 정확한 등식이다(반올림 없음).

  시간 쪽: 시공간 그래프의 마디는 (i, n)이고 시간 변이 (i, n) -> (j, n+1)이다.
      공간 먼저(선 방법 + 후방 오일러)   (u^{n+1}_i - u^n_i)/tau = (L u^{n+1})_i
      시간 먼저(로테: 걸음마다 타원 문제) (I - tau A) u^{n+1} = u^n 을 세운 뒤, 걸음마다 그 걸음의 격자로 A를 이산
```

```
  고정 격자이면 두 순서의 시공간 행렬이 정확히 같고(commute_check), 시간 먼저에서 걸음마다 격자가 바뀌면
  시간 변의 무게가 이전 격자에서 새 격자로의 P1 보간 계수가 된다(spacetime_rothe_moving). 어느 순서든 그래프가 된다.
"""
import sympy as sp
import numpy as np

R = sp.Rational


class Graph:
    """가중 그래프 G = (마디 위치 pos, 마디 무게 V, 변 무게 T).  T는 {(i, j): T_ij} (i < j), 무향."""
    def __init__(self, pos, V, T, name=""):
        self.pos, self.V, self.T, self.name = list(pos), list(V), dict(T), name

    @property
    def n(self): return len(self.V)


# ---------------------------------------------------------------- 그림 -> 수식
def matrix_from_graph(G):
    """식 (G): L_ij = T_ij / V_i (j ~ i),  L_ii = -sum_j T_ij / V_i.   정확한 유리수 행렬."""
    L = sp.zeros(G.n, G.n)
    for (i, j), t in G.T.items():
        L[i, j] += t / G.V[i]; L[j, i] += t / G.V[j]
        L[i, i] -= t / G.V[i]; L[j, j] -= t / G.V[j]
    return L


def stencil_formula(G, i, u=None):
    """i행의 식을 기호로: (1/V_i) * sum_j T_ij (u_j - u_i).  u: 기호 목록(없으면 u_0, u_1, ...)."""
    u = u or sp.symbols(f"u_0:{G.n}")
    s = sum((t * (u[j if a == i else a] - u[i]) for (a, j), t in G.T.items() if i in (a, j)), sp.Integer(0))
    return sp.simplify(s / G.V[i])


# ---------------------------------------------------------------- 수식 -> 그림
def graph_from_matrix(L, V, pos=None, name=""):
    """L이 식 (G)의 꼴인지 정확히 확인하고 그래프로 바꾼다: V_i L_ij = V_j L_ji(대칭), 행합 0.
    아니면 ValueError(그 수식은 이 그래프 꼴로 그릴 수 없다는 반례)."""
    n = L.shape[0]; T = {}
    for i in range(n):
        if sp.simplify(sum(L[i, j] for j in range(n))) != 0: raise ValueError(f"{i}행의 합이 0이 아니다")
        for j in range(i + 1, n):
            a, b = sp.simplify(V[i] * L[i, j]), sp.simplify(V[j] * L[j, i])
            if a != b: raise ValueError(f"V_{i} L_{i}{j} = {a} != {b} = V_{j} L_{j}{i}")
            if a != 0: T[(i, j)] = a
    return Graph(pos if pos is not None else list(range(n)), list(V), T, name)


def structure(G):
    """명제 4.1을 그래프에서 읽는다: (메츨러, 행합 0, V L 대칭) — 모두 정확한 판정."""
    L = matrix_from_graph(G); n = G.n
    metz = all(L[i, j] >= 0 for i in range(n) for j in range(n) if i != j)
    rows = all(sp.simplify(sum(L[i, j] for j in range(n))) == 0 for i in range(n))
    VL = sp.diag(*G.V) * L
    return metz, rows, sp.simplify(VL - VL.T) == sp.zeros(n, n)


# ---------------------------------------------------------------- 세 공간 이산을 그래프로
def fdm_1d(N, Lx=1):
    """FDM(꼭짓점, 영 유량 유령점): (u_{i+1} - 2u_i + u_{i-1})/h^2.  그래프: 사슬, T = 1/h, V = (h/2, h, ..., h/2)."""
    h = R(Lx, N - 1)
    return Graph([i * h for i in range(N)], [h / 2] + [h] * (N - 2) + [h / 2], {(i, i + 1): 1 / h for i in range(N - 1)}, "FDM")


def fvm_1d(N, Lx=1):
    """FVM(칸 중심, 2점 플럭스): (1/h)[(u_{i+1} - u_i)/h - (u_i - u_{i-1})/h].  그래프: 칸 = 마디, 면 = 변, T = 1/h, V = h."""
    h = R(Lx, N)
    return Graph([h * i + h / 2 for i in range(N)], [h] * N, {(i, i + 1): 1 / h for i in range(N - 1)}, "FVM")


def fem_p1_1d(x):
    """FEM P1(집중 질량): L = -M_l^{-1} K, K_e = [1, -1; -1, 1]/h_e.  그래프: 요소 = 변, T = 1/h_e, V_i
```

```
    = (h_{i-1} + h_i)/2."""
    x = [sp.nsimplify(v) for v in x]; N = len(x); V = [sp.Integer(0)] * N; T = {}
    for e in range(N - 1):
        he = x[e + 1] - x[e]; T[(e, e + 1)] = 1 / he; V[e] += he / 2; V[e + 1] += he / 2
    return Graph(x, V, T, "FEM P1")


def fdm_2d(n, m=None):
    """FDM 5점 스텐실(영 유량, 꼭짓점): 격자 그래프, T = 1(h = 1로 정규화), V = 사다리꼴 무게의 곱."""
    m = m or n; idx = lambda i, j: i * m + j; T = {}
    wt = lambda k, K: R(1, 2) if k in (0, K - 1) else sp.Integer(1)
    for i in range(n):
        for j in range(m):
            if i + 1 < n: T[(idx(i, j), idx(i + 1, j))] = sp.Integer(1) * (wt(j, m))
            if j + 1 < m: T[(idx(i, j), idx(i, j + 1))] = sp.Integer(1) * (wt(i, n))
    V = [wt(i, n) * wt(j, m) for i in range(n) for j in range(m)]
    return Graph([(j, i) for i in range(n) for j in range(m)], V, T, "FDM 5점")


def fem_p1_tri(n, m=None):
    """2차원 P1(직각 삼각형 망, 집중 질량): 강성 K_ij = -(cot a + cot b)/2 (변 ij의 맞은편 두 각).
    그래프: 망의 변 = 변, T_ij = (cot a + cot b)/2, V_i = (마디를 둘러싼 삼각형 넓이 합)/3."""
    m = m or n; idx = lambda i, j: i * m + j; T = {}; V = [sp.Integer(0)] * (n * m)
    P = {idx(i, j): (sp.Integer(j), sp.Integer(i)) for i in range(n) for j in range(m)}
    def cot(o, a, b):                       # 마디 o에서 a, b를 보는 각의 코탄젠트 = (내적)/(외적의 크기)
        ax, ay = P[a][0] - P[o][0], P[a][1] - P[o][1]; bx, by = P[b][0] - P[o][0], P[b][1] - P[o][1]
        return (ax * bx + ay * by) / abs(ax * by - ay * bx)
    for i in range(n - 1):
        for j in range(m - 1):
            a, b, c, d = idx(i, j), idx(i, j + 1), idx(i + 1, j), idx(i + 1, j + 1)
            for tri in ((a, b, d), (a, d, c)):                # 사각형을 대각선 a–d로 나눈다
                for k in range(3):
                    p, q, o = tri[k], tri[(k + 1) % 3], tri[(k + 2) % 3]
                    key = (min(p, q), max(p, q)); T[key] = T.get(key, 0) + cot(o, p, q) / 2
                for v in tri: V[v] += R(1, 2) / 3
    T = {k: v for k, v in T.items() if v != 0}                # 대각선 변은 맞은편 두 각이 직각이라 무게 0
    return Graph([P[k] for k in range(n * m)], V, T, "FEM P1 삼각형")


# ---------------------------------------------------------------------- 시공간 그래프: 순서와 상관없이
def spacetime_mol_be(G, tau, steps):
    """공간 먼저: 반이산계 u' = L u를 후방 오일러로.  n+1층의 식 (u^{n+1} - u^n)/tau - L u^{n+1} = 0.
    시공간 행렬 B(블록 하삼각): 대각 블록 I/tau - L, 아래 블록 -I/tau.  시간 변 (i, n) -> (i, n+1)의 무게 1/tau."""
    L = matrix_from_graph(G); n = G.n; I = sp.eye(n)
    B = sp.zeros(n * steps, n * (steps + 1))
    for k in range(steps):
        B[k * n:(k + 1) * n, k * n:(k + 1) * n] = -I / tau
        B[k * n:(k + 1) * n, (k + 1) * n:(k + 2) * n] = I / tau - L
    return B


def spacetime_rothe(grid_of_step, tau, steps):
    """시간 먼저(로테): 먼저 시간만 이산해 걸음마다 타원 문제 (I - tau A) u^{n+1} = u^n 을 세우고(A는 미분 연산자 그대로),
    그 다음 걸음마다 그 걸음의 격자 G_{n+1} = grid_of_step(n+1)로 A를 식 (G)로 이산한다.
    격자가 바뀌면 u^n을 새 격자로 옮기는 P1 보간 행렬 P_n이 시간 변의 무게가 된다:
        (u^{n+1} - P_n u^n)/tau = L_{n+1} u^{n+1}.
    반환: 블록 행렬 B와 각 층의 격자 목록."""
    Gs = [grid_of_step(k) for k in range(steps + 1)]; sizes = [g.n for g in Gs]; off = np.concatenate([[0], np.cumsum(sizes)])
    B = sp.zeros(sum(sizes[1:]), sum(sizes)); r = 0
    for k in range(steps):
        Gn, Gm = Gs[k], Gs[k + 1]; P = p1_interp(Gn.pos, Gm.pos); L = matrix_from_graph(Gm); I = sp.eye(Gm.n)
        B[r:r + Gm.n, off[k]:off[k + 1]] = -P / tau
        B[r:r + Gm.n, off[k + 1]:off[k + 2]] = I / tau - L
        r += Gm.n
    return B, Gs


def p1_interp(xs, ys):
    """옛 마디 xs의 P1 보간을 새 마디 ys에서 읽는 행렬: (P u)_a = sum_i phi_i(y_a) u_i (정확한 유리수)."""
    P = sp.zeros(len(ys), len(xs))
    for a, y in enumerate(ys):
        for i in range(len(xs) - 1):
            if xs[i] <= y <= xs[i + 1]:
```

```
                s = (y - xs[i]) / (xs[i + 1] - xs[i]); P[a, i] += 1 - s; P[a, i + 1] += s; break
    return P


def commute_check(G, tau=R(1, 10), steps=3):
    """고정 격자: 공간 먼저(MOL + 후방 오일러)와 시간 먼저(로테 + 같은 격자)의 시공간 행렬이 정확히 같은가."""
    B1 = spacetime_mol_be(G, tau, steps); B2, _ = spacetime_rothe(lambda k: G, tau, steps)
    return B1 == B2


# ---------------------------------------------------------------- 그림
def draw_graph(ax, G, color, label_edges=True, y=0.0, fmt=None, node_label=None, ms=5, lw_scale=1.0):
    """마디 = 점(크기는 V_i에 비례), 변 = 선(굵기는 T_ij에 비례), 변 위 글자 = T_ij.  1차원은 높이 y에 그린다."""
    pos = [(float(p), y) if not isinstance(p, tuple) else (float(p[0]), float(p[1])) for p in G.pos]
    tmax = max(float(t) for t in G.T.values())
    for (i, j), t in G.T.items():
        (x0, y0), (x1, y1) = pos[i], pos[j]
        ax.plot([x0, x1], [y0, y1], color=color, lw=lw_scale * (0.6 + 1.6 * float(t) / tmax), zorder=1, solid_capstyle="butt")
        if label_edges:
            vert = abs(x1 - x0) < 1e-12
            ax.text((x0 + x1) / 2 + (0.07 if vert else 0), (y0 + y1) / 2 + (0 if vert else 0.06), fmt(t) if fmt else sp.latex(t),
                    fontsize=5.6, ha="left" if vert else "center", va="center" if vert else "bottom", color=color)
    vmax = max(float(v) for v in G.V)
    for k, (px, py) in enumerate(pos):
        ax.plot(px, py, "o", ms=ms * (0.55 + 0.45 * float(G.V[k]) / vmax), color="white", mec=color, mew=1.1, zorder=2)
        if node_label: ax.text(px, py - 0.09, node_label(k), fontsize=5.6, ha="center", va="top", color="#444444")
```

space_check.py — 공간 스킴의 에러 검사: 주장의 그래프 상수와 정확한 유리수 비교, 보존 규칙, 층 사이 보간, 비선형 플럭스(4.5절)

```
"""space_check: 공간 이산의 에러 검사 — 검사 순서의 첫 단계(공간 -> 시간 -> 정확도).

공간 연산자는 격자·망 그래프 위의 상수 행렬이다(4.4절 식 (G)):
    (L u)_i = (1 / V_i) * sum_{j ~ i} T_ij (u_j - u_i)
대상이 준 것은 저장된 부동소수 행렬 L(또는 u -> L u 를 계산하는 블랙박스 함수; 단위 벡터를 넣어 열을 뽑는다)과
마디 위치 x이다.  주장(어떤 공간 이산인가)이 있으면 그 이산의 상수 (V*, T*)를 마디 위치에서 정확한 유리수로 다시
만들어 L*을 얻는다.  저장된 부동소수와 마디 위치는 그 자체로 정확한 유리수이므로 아래 비교에는 반올림이 없다.

규칙(반례가 나면 공간 구현 에러, 모두 통과하면 증명)
  R1 관계식(주장 필요)  |L_ij - L*_ij| <= g * A_i,   A_i = sum_j |L*_ij| (그 행의 크기)
                        g = gam(K_SP): 저장값 하나를 만드는 데 허용한 연산 수 K_SP의 표준 모형 상한(정의).
                        조립에서 서로 지워져 정확히 0이어야 할 항목도 반올림 찌꺼기를 가지므로 허용치는 행의 크기로 잰다
  R2 상수 보존(주장 불필요)  (L 1)_i = 0 :  |sum_j L_ij| <= g * sum_j |L_ij|
                        영 유량 경계의 확산 이산은 상수를 정확히 보존해야 한다(행합 0, 식 (G))
  R3 1차 보존(주장 불필요)  내부 마디에서 (L x)_i = 0 :  |sum_j L_ij x_j| <= g * sum_j |L_ij||x_j|
                        일관된 2계 미분 이산은 1차 함수를 0으로 보내야 한다
  R4 층 사이 보간(시간 먼저, 격자가 바뀔 때)  P_n의 각 행의 합 = 1, 항목 >= 0, 주장(P1 보간)이면 항목별 R1과 같은 비교
각 반례는 (층, 마디 i, 이웃 j, 규칙, 저장값, 정확한 기대값, 초과량)으로 남는다.  이것이 육하원칙의 어디서·왜이다.
처방(어떻게)은 그 자리에 정확한 기대값을 넣은 행렬로 다시 검사해 통과하는지를 확인한 것만 적는다."""
from fractions import Fraction as Fr
import numpy as np

U = Fr(1, 2 ** 53)
K_SP = 64                       # 저장된 공간 연산자의 항목 하나를 만드는 데 허용한 부동소수 연산 수(정의)


def gamF(k):
    return k * U / (1 - k * U)


def exact(v):
    """부동소수 하나를 그 값 그대로의 유리수로(반올림 없음)."""
    return Fr(float(v))
```

```
34
35
36  # ------------------------------------------------------------------ 주장한 이산의 상수 (V*, T*) -> L* (
    정확한 유리수)
37  def axis_graph(kind, x):
38      """1차원 축 하나의 (V, T).  'FDM'(꼭짓점, 영 유량 유령점, 균일), 'FVM'(칸 중심, 2점 플럭스, 균일),
39      'FEM P1'(집중 질량, 아무 마디)."""
40      xe = [exact(v) for v in x]; N = len(xe)
41      if kind == "FEM P1":
42          V = [Fr(0)] * N; T = {}
43          for e in range(N - 1):
44              he = xe[e + 1] - xe[e]; T[(e, e + 1)] = 1 / he; V[e] += he / 2; V[e + 1] += he / 2
45          return V, T
46      h = (xe[-1] - xe[0]) / (N - 1)
47      V = ([h / 2] + [h] * (N - 2) + [h / 2]) if kind == "FDM" else [h] * N
48      return V, {(i, i + 1): 1 / h for i in range(N - 1)}
49
50
51  def grid_index(shape):
52      """d차원 텐서 격자의 번호: flat = sum_k i_k * prod_{l<k} n_l (축 0이 가장 빨리 돈다)."""
53      strides = [1]
54      for n_ in shape[:-1]: strides.append(strides[-1] * n_)
55      return strides
56
57
58  def claim_graph(kind, x):
59      """2계 편미분 -div(grad u)의 공간 이산을 그래프 상수 (V, T)로(정확한 유리수).
60      x: 1차원 배열(1차원) 또는 축마다의 1차원 좌표 튜플 (x_0, ..., x_{d-1})(d차원 텐서 격자, 차원 제한 없
    음).
61      kind 'FDM' | 'FVM' | 'FEM P1': 축별 1차원 이산의 크로네커 합.  그래프로는
62          V(마디) = prod_k V_k(i_k),   T(축 k 방향 변) = T_k(변) * prod_{l != k} V_l(i_l)
63      (d차원 FDM은 2d+1점 스텐실, FVM은 직육면체 칸의 2점 플럭스, FEM은 축별 P1 집중 질량의 텐서곱).
64      kind 'FEM P1 삼각형': 2차원, 각 사각형을 (x_j, y_i)-(x_{j+1}, y_{i+1}) 대각선으로 나눈 삼각형 망, 코
    탄젠트 무게."""
65      if not isinstance(x, tuple): return axis_graph(kind, x)
66      if kind == "FEM P1 삼각형": return _tri_graph(x)
67      ax = [axis_graph(kind, xk) for xk in x]; shape = [len(xk) for xk in x]; st = grid_index(shape); d =
    len(x)
68      n = int(np.prod(shape)); V = [Fr(1)] * n; T = {}
69      for f in range(n):
70          ii = [(f // st[k]) % shape[k] for k in range(d)]
71          for k in range(d): V[f] *= ax[k][0][ii[k]]
72          for k in range(d):
73              if ii[k] + 1 < shape[k]:
74                  t = ax[k][1][(ii[k], ii[k] + 1)]
75                  for l in range(d):
76                      if l != k: t *= ax[l][0][ii[l]]
77                  T[(f, f + st[k])] = t
78      return V, T
79
80
81  def _tri_graph(x):
82      xs, ys = [exact(v) for v in x[0]], [exact(v) for v in x[1]]; n, m = len(ys), len(xs)
83      idx = lambda i, j: i * m + j; T = {}
84      P = {idx(i, j): (xs[j], ys[i]) for i in range(n) for j in range(m)}; V = [Fr(0)] * (n * m)
85      def cot(o, a, b):
86          ax_, ay = P[a][0] - P[o][0], P[a][1] - P[o][1]; bx, by = P[b][0] - P[o][0], P[b][1] - P[o][1]
87          return (ax_ * bx + ay * by) / abs(ax_ * by - ay * bx)
88      for i in range(n - 1):
89          for j in range(m - 1):
90              a, b, c, d = idx(i, j), idx(i, j + 1), idx(i + 1, j), idx(i + 1, j + 1)
91              area = (xs[j + 1] - xs[j]) * (ys[i + 1] - ys[i]) / 2
92              for tri in ((a, b, d), (a, d, c)):
93                  for k in range(3):
94                      p, q, o = tri[k], tri[(k + 1) % 3], tri[(k + 2) % 3]
95                      key = (min(p, q), max(p, q)); T[key] = T.get(key, Fr(0)) + cot(o, p, q) / 2
96                  for v in tri: V[v] += area / 3
97      return V, {k: v for k, v in T.items() if v != 0}
98
99
100 def node_coords(x):
101     """마디마다 좌표(정확한 유리수 튜플)와 축마다 그 마디가 경계에 있는지."""
102     if not isinstance(x, tuple): x = (x,)
103     xe = [[exact(v) for v in xk] for xk in x]; shape = [len(xk) for xk in xe]; st = grid_index(shape); n
     = int(np.prod(shape))
104     C, B = [], []
105     for f in range(n):
106         ii = [(f // st[k]) % shape[k] for k in range(len(x))]
107         C.append(tuple(xe[k][ii[k]] for k in range(len(x)))); B.append(tuple(ii[k] in (0, shape[k] - 1)
    for k in range(len(x))))
```

```
    return C, B


def matrix_from_VT(V, T):
    """식 (G): 정확한 유리수 행렬(리스트의 리스트)."""
    n = len(V); L = [[Fr(0)] * n for _ in range(n)]
    for (i, j), t in T.items():
        L[i][j] += t / V[i]; L[j][i] += t / V[j]; L[i][i] -= t / V[i]; L[j][j] -= t / V[j]
    return L


def p1_interp_exact(xs, ys):
    xs = [exact(v) for v in xs]; ys = [exact(v) for v in ys]; P = [[Fr(0)] * len(xs) for _ in ys]
    for a, y in enumerate(ys):
        for i in range(len(xs) - 1):
            if xs[i] <= y <= xs[i + 1]:
                s = (y - xs[i]) / (xs[i + 1] - xs[i]); P[a][i] += 1 - s; P[a][i + 1] += s; break
    return P


# ---------------------------------------------------------------- 검사
def as_matrix(L, n=None):
    """저장된 행렬, 또는 u -> L u 블랙박스(단위 벡터로 열을 뽑는다)."""
    if callable(L):
        cols = []
        for j in range(n):
            e = np.zeros(n); e[j] = 1.0; cols.append(np.asarray(L(e), float))
        return np.array(cols).T
    return np.asarray(L, float)


def check(L, x, kind=None, interior=None, level=None, pos=None):
    """한 층의 공간 연산자 검사.  반환 dict(ok, viol=[...], n_checked, rules)."""
    Lf = as_matrix(L, len(x) if not isinstance(x, tuple) else int(np.prod([len(xk) for xk in x])))
    n = Lf.shape[0]; g = gamF(K_SP); viol = []
    LE = [[exact(Lf[i, j]) for j in range(n)] for i in range(n)]
    # R1 관계식
    if kind is not None:
        Ls = matrix_from_VT(*claim_graph(kind, x))
        for i in range(n):
            A = sum(abs(Ls[i][j]) for j in range(n))
            for j in range(n):
                if LE[i][j] == 0 and Ls[i][j] == 0: continue
                d = abs(LE[i][j] - Ls[i][j])
                if d > g * A:
                    viol.append(dict(level=level, i=i, j=j, rule="관계식", stored=float(Lf[i, j]), expec
ted=Ls[i][j],
                                     excess=float(d - g * A)))
    # R2 상수 보존
    for i in range(n):
        s_ = sum(LE[i]); bnd = g * sum(abs(v) for v in LE[i])
        if abs(s_) > bnd: viol.append(dict(level=level, i=i, j=None, rule="상수 보존(행합 0)", stored=fl
oat(s_), expected=Fr(0), excess=float(abs(s_) - bnd)))
    C, Bd = node_coords(x)
    # R3 1차 보존: 축 k의 좌표 함수 x_k를 그 축에서 경계가 아닌 마디에서 검사(차원 제한 없음)
    for k in range(len(C[0])):
        for i in range(n):
            if Bd[i][k]: continue
            s_ = sum(LE[i][j] * C[j][k] for j in range(n)); bnd = g * sum(abs(LE[i][j]) * abs(C[j][k]) f
or j in range(n))
            if abs(s_) > bnd: viol.append(dict(level=level, i=i, j=None, rule="1차 보존", stored=float(s
_), expected=Fr(0), excess=float(abs(s_) - bnd)))
    return dict(ok=not viol, viol=viol, n=n, rules=("관계식, " if kind else "") + "상수 보존, 1차 보존",
 kind=kind)


def check_interp(P, xs, ys, level, claimed=True):
    """R4: 층 사이 보간 행렬(시간 먼저, 격자가 바뀔 때)."""
    P = np.asarray(P, float); g = gamF(K_SP); viol = []
    Ps = p1_interp_exact(xs, ys) if claimed else None
    for a in range(P.shape[0]):
        row = [exact(v) for v in P[a]]; s = sum(row)
        if abs(s - 1) > g * sum(abs(v) for v in row): viol.append(dict(level=level, i=a, j=None, rule="
보간 행합 1", stored=float(s), expected=Fr(1), excess=float(abs(s - 1))))
        for j, v in enumerate(row):
            if v < 0: viol.append(dict(level=level, i=a, j=j, rule="보간 무게 >= 0", stored=float(v), ex
pected=Fr(0), excess=float(-v), xj=float(xs[j])))
            if Ps is not None and abs(v - Ps[a][j]) > g * abs(Ps[a][j]):
                viol.append(dict(level=level, i=a, j=j, rule="보간 관계식(P1)", stored=float(v), expecte
d=Ps[a][j], excess=float(abs(v - Ps[a][j])), xj=float(xs[j])))
```

```
    return dict(ok=not viol, viol=viol, n=P.shape[0])


def check_flux(Fs, x, kind, coef, states, level=None):
    """비선형·변수 계수 2계 편미분 -div(c(x, u) grad u)의 공간 이산 검사.  Fs: u -> 공간 항(블랙박스 함수, 부동소수).
    주장(식 (G)에 계수를 곱한 꼴):  F*_i(u) = (1/V_i) sum_{j ~ i} T_ij c(x_ij, u_ij) (u_j - u_i),
        x_ij = 변의 가운데,  u_ij = (u_i + u_j)/2  (coef는 유리수에서 정확히 계산되는 함수: 다항식·유리식)
    상태 u마다(시험 상태 states, 예: 실제 궤적의 상태) 정확한 유리수로 F*를 계산해 비교한다(반올림 없음).
      R1' 관계식:   |Fs_i - F*_i| <= g * (1/V_i) sum_j |T_ij c_ij| (|u_i| + |u_j|)
      R2' 질량 보존: |sum_i V_i Fs_i| <= g * sum_i V_i (|Fs_i| + 허용치_i)   (대칭 플럭스면 정확히 0; 영유량 경계)
      R3' 상수:     Fs(상수) = 0  (주장 불필요)"""
    V, T = claim_graph(kind, x); C, _ = node_coords(x); n = len(V); g = gamF(K_SP); viol = []
    nb = [[] for _ in range(n)]
    for (i, j), t in T.items(): nb[i].append((j, t)); nb[j].append((i, t))
    for si, u in enumerate(list(states) + [np.full(n, 0.7)]):
        uf = np.asarray(u, float); Fv = np.asarray(Fs(uf.copy()), float); ue = [exact(v) for v in uf]; Fe = [exact(v) for v in Fv]
        const = si == len(states); tot = Fr(0); totb = Fr(0)
        for i in range(n):
            Fstar = Fr(0); A = Fr(0)
            for j, t in nb[i]:
                xm = tuple((C[i][k] + C[j][k]) / 2 for k in range(len(C[i]))); cij = coef(xm, (ue[i] + ue[j]) / 2)
                Fstar += t * cij * (ue[j] - ue[i]); A += abs(t * cij) * (abs(ue[i]) + abs(ue[j]))
            Fstar /= V[i]; A /= V[i]; d = abs(Fe[i] - Fstar)
            if d > g * A:
                viol.append(dict(level=level, state=("상수" if const else si), i=i, j=None, rule=("상수(Fs(상수) = 0)" if const else "관계식(비선형 플럭스)"),
                                 stored=float(Fv[i]), expected=Fstar, excess=float(d - g * A)))
            tot += V[i] * Fe[i]; totb += V[i] * (abs(Fe[i]) + A)
        if abs(tot) > g * totb:
            viol.append(dict(level=level, state=si, i=None, j=None, rule="질량 보존(sum V_i F_i = 0)", stored=float(tot), expected=Fr(0), excess=float(abs(tot) - g * totb)))
    return dict(ok=not viol, viol=viol, n=n, rules="비선형 플럭스 관계식, 질량 보존, 상수", kind=kind + " + 계수 c(x, u)")


def patched(L, viol, rule="관계식"):
    """처방: 반례가 난 항목에 정확한 기대값을 넣은 행렬(rule = '관계식' 또는 '보간 관계식(P1)')."""
    L2 = np.array(L, float, copy=True)
    for v in viol:
        if v["rule"] == rule: L2[v["i"], v["j"]] = float(v["expected"])
    return L2


def fmtq(q):
    """정확한 유리수를 읽기 좋게: 분모가 작으면 분수, 아니면 유효숫자 12자리(값 자체는 정확한 유리수로 계산했다)."""
    q = Fr(q)
    return str(q) if q.denominator <= 10 ** 6 else f"{float(q):.12g}"


def _where(v, x):
    def at(k):
        if x is None: return ""
        if isinstance(x, tuple):
            shape = [len(xk) for xk in x]; st = grid_index(shape); ii = [(k // st[a]) % shape[a] for a in range(len(x))]
            nm = "xyz"[:len(x)] if len(x) <= 3 else [f"x{a}" for a in range(len(x))]
            return f"({', '.join(nm)}) = ({', '.join(f'{float(x[a][ii[a]]):.3g}' for a in range(len(x)))})"
        return f"x = {float(x[k]):.4g}"
    if v["i"] is None:
        return ("" if v["level"] is None else f"층 {v['level']}, ") + (f"상태 {v['state']}, " if v.get("state") is not None else "") + "영역 전체(합)"
    p = at(v["i"])
    if v["j"] is None: q = ""
    elif "xj" in v: q = f", 앞 층 마디 {v['j']}(x = {v['xj']:.4g})"
    else: q = ", 대각 항" if v["j"] == v["i"] else f", 이웃 마디 {v['j']}({at(v['j'])})"
    lv = ("" if v["level"] is None else f"층 {v['level']}, ") + (f"상태 {v['state']}, " if v.get("state") is not None else "")
    return f"{lv}마디 {v['i']}" + (f"({p})" if p else "") + q


def report(res, x=None, name="대상 공간 이산", fix=None):
    """육하원칙 보고(공간 단계).  fix: 처방 재검사 결과 dict(있으면 어떻게에 적는다)."""
```

```
    rep = {}; vs = sorted(res["viol"], key=lambda v: -v["excess"])
    if res["ok"]:
        rep["판정"] = dict(status="증명", text=f"공간 에러 없음: 검사한 {res['n']}개 마디 모두에서 규칙({res['rules']})이 허용치 gam({res.get('gk', K_SP)}) 안이다")
        rep["누가"] = dict(status="증명" if res["kind"] else "정의", text=(f"{res['kind']}의 관계식과 일치" if res["kind"] else "주장 없음(규칙 R2–R3만 검사)"))
        return rep
    v0 = vs[0]
    rep["판정"] = dict(status="반례", text=f"공간 구현 에러: {name}에서 규칙 위반 {len(vs)}건(반례)")
    rels = [v for v in vs if "관계식" in v["rule"]]
    rep["누가"] = dict(status="반례", text=(f"주장한 이산({res['kind']}{', 층 사이 P1 보간' if any(v['rule'].startswith('보간') for v in rels) else ''})이 아니다" if rels else "주장과 무관하게 일관성 규칙을 어긴다"))
    rep["무엇을"] = dict(status="반례", text=f"가장 큰 어긋남: {v0['rule']}, 저장값 {v0['stored']:.10g}, 정확한 기대값 {fmtq(v0['expected'])}, 허용치를 {v0['excess']:.3e} 넘는다")
    lv = sorted({v["level"] for v in vs if v["level"] is not None})
    st_ = sorted({str(v["state"]) for v in vs if v.get("state") is not None})
    rep["언제"] = dict(status="반례" if (lv or st_) else "정의", text=(f"시간 층 {', '.join(map(str, lv))}" if lv else
                    (f"시험 상태 {', '.join(st_)}에서(비선형 연산자는 상태마다 다르다)" if st_ else "시간과 무관(모든 걸음에 같은 공간 연산자)")))
    rep["어디서"] = dict(status="반례", text="; ".join(f"{_where(v, x)} [{v['rule']}]" for v in vs[:4]) + (f" 외 {len(vs) - 4}건" if len(vs) > 4 else ""))
    rules = sorted({v["rule"] for v in vs})
    why = {"관계식": "항목이 주장한 이산의 상수(V, T)와 다르다", "상수 보존(행합 0)": "상수를 보존하지 않아 질량이 새거나 생긴다",
           "1차 보존": "1차 함수를 0으로 보내지 않아 2계 미분과 일관하지 않다", "보간 행합 1": "층 사이 보간이 상수를 옮기지 못한다",
           "보간 무게 >= 0": "보간이 단조가 아니다", "보간 관계식(P1)": "층 사이 보간이 주장한 P1 보간이 아니다",
           "관계식(비선형 플럭스)": "면 플럭스가 주장한 c(x_ij, (u_i + u_j)/2) T_ij (u_j - u_i)와 다르다",
           "질량 보존(sum V_i F_i = 0)": "면 플럭스가 양쪽 마디에서 같지 않아(비대칭) 질량이 새거나 생긴다",
           "상수(Fs(상수) = 0)": "상수 상태에서 공간 항이 0이 아니다"}
    rep["왜"] = dict(status="반례", text="; ".join(f"{r}: {why.get(r, r)}" for r in rules))
    fixes = [f"L[{v['i']},{v['j']}] = {fmtq(v['expected'])}" for v in vs if v["rule"] == "관계식"]
    if fix is not None and fixes:
        rep["어떻게"] = dict(status="증명" if fix["ok"] else "반례",
                             text=("반례 자리에 정확한 기대값을 넣어 다시 검사하면 통과한다(처방 확인): " if fix["ok"] else "정확한 기대값을 넣어도 통과하지 않는다(반례): ")
                             + ", ".join(fixes[:4]) + (f" 외 {len(fixes) - 4}건" if len(fixes) > 4 else ""))
    elif any(v["rule"].startswith("관계식(") and "비선형 플럭스" not in v["rule"] for v in vs):
        rep["어떻게"] = dict(status=("증명" if fix is not None and fix["ok"] else "정의"),
                             text="반례가 난 마디의 공식 계산(주장한 식의 각 항)을 정확한 기대값과 맞게 고친다: " + ", ".join(f"마디 {v['i']}의 기대값 {fmtq(v['expected'])}" for v in vs[:3])
                             + ("; 고친 코드로 다시 검사하면 통과한다(처방 확인)" if fix is not None and fix["ok"] else ""))
    elif any("비선형 플럭스" in v["rule"] or "질량 보존" in v["rule"] for v in vs):
        rep["어떻게"] = dict(status=("증명" if fix is not None and fix["ok"] else "정의"),
                             text="면마다 계수를 c(x_ij, (u_i + u_j)/2)로 한 번만 계산해 양쪽 마디에 같은 플럭스로 더한다"
                             + ("; 그렇게 고친 코드로 다시 검사하면 통과한다(처방 확인)" if fix is not None and fix["ok"] else ""))
    elif any(v["rule"].startswith("보간") for v in vs) and fix is not None:
        rep["어떻게"] = dict(status="증명" if fix["ok"] else "반례", text=("반례 자리에 P1 보간 무게를 넣어 다시 검사하면 통과한다(처방 확인): " if fix["ok"] else "고쳐도 통과하지 않는다(반례): ")
                             + ", ".join([f"P[{v['i']},{v['j']}] = {fmtq(v['expected'])}" for v in vs if v['rule'] == '보간 관계식(P1)'][:4]))
    elif any(v["rule"].startswith("보간") for v in vs):
        rep["어떻게"] = dict(status="정의", text="층 사이 보간을 P1 보간 무게 (1 - s, s)로 바꾼다: " + ", ".join([f"P[{v['i']},{v['j']}] = {fmtq(v['expected'])}" for v in vs if v['rule'] == '보간 관계식(P1)'][:4]))
    else:
        rep["어떻게"] = dict(status="정의", text="주장한 이산이 없어 정확한 값은 처방하지 않는다; 위반한 행을 고친 뒤 다시 검사한다")
    return rep


# ============================================================== 2계 이상: 고계 편미분 (4.6절)
def mat_power_exact(Ls, p):
    """정확한 유리수 행렬의 p제곱(희소 곱)."""
    n = len(Ls); nz = [[(j, v) for j, v in enumerate(row) if v != 0] for row in Ls]; P = Ls
    for _ in range(p - 1):
        Pn = [[Fr(0)] * n for _ in range(n)]
        for i in range(n):
            acc = {}
            for k, a in nz[i]:
                for j, b in enumerate(P[k]):
```

```
                    if b != 0: acc[j] = acc.get(j, Fr(0)) + a * b
            for j, v in acc.items(): Pn[i][j] = v
        P = Pn
    return P


def check_power(L, x, kind, p, level=None):
    """2p계 편미분 (-Δ)^p 의 이산이 그래프 연산자의 p제곱 L*^p 인지 검사한다(쌍조화 Δ² 는 p = 2).
    R1(관계식)은 L*^p 와의 항목별 비교(허용치는 행의 크기, K_SP 정의 그대로), R2 행합 0, R3 1차 보존(축마다 경계에서 p칸 안쪽)."""
    Lf = as_matrix(L, len(x) if not isinstance(x, tuple) else int(np.prod([len(xk) for xk in x])))
    n = Lf.shape[0]; g = gamF(K_SP * p); viol = []
    LE = [[exact(Lf[i, j]) for j in range(n)] for i in range(n)]
    Ls = mat_power_exact(matrix_from_VT(*claim_graph(kind, x)), p)
    for i in range(n):
        A = sum(abs(v) for v in Ls[i])
        for j in range(n):
            if LE[i][j] == 0 and Ls[i][j] == 0: continue
            d = abs(LE[i][j] - Ls[i][j])
            if d > g * A: viol.append(dict(level=level, i=i, j=j, rule="관계식", stored=float(Lf[i, j]), expected=Ls[i][j], excess=float(d - g * A)))
    for i in range(n):
        s_ = sum(LE[i]); bnd = g * sum(abs(v) for v in LE[i])
        if abs(s_) > bnd: viol.append(dict(level=level, i=i, j=None, rule="상수 보존(행합 0)", stored=float(s_), expected=Fr(0), excess=float(abs(s_) - bnd)))
    C, _ = node_coords(x); shape = [len(xk) for xk in (x if isinstance(x, tuple) else (x,))]; st = grid_index(shape)
    for k in range(len(C[0])):
        for i in range(n):
            ik = (i // st[k]) % shape[k]
            if ik < p or ik > shape[k] - 1 - p: continue
            s_ = sum(LE[i][j] * C[j][k] for j in range(n)); bnd = g * sum(abs(LE[i][j]) * abs(C[j][k]) for j in range(n))
            if abs(s_) > bnd: viol.append(dict(level=level, i=i, j=None, rule="1차 보존", stored=float(s_), expected=Fr(0), excess=float(abs(s_) - bnd)))
    return dict(ok=not viol, viol=viol, n=n, rules=f"관계식(L^{p}), 상수 보존, 1차 보존", kind=f"{kind}의 {p}제곱(2·{p}계)", gk=K_SP * p)


class EV:
    """정확한 값과 그 계산의 크기(반올림 허용치의 바탕)를 함께 나르는 벡터: 고계·비선형 공식의 기대값을 만든다."""
    def __init__(s, v, m): s.v, s.m = list(v), list(m)
    @staticmethod
    def of(u): return EV(u, [abs(a) for a in u])
    def __add__(s, o): o = o if isinstance(o, EV) else EV([o] * len(s.v), [abs(o)] * len(s.v)); return EV([a + b for a, b in zip(s.v, o.v)], [a + b for a, b in zip(s.m, o.m)])
    def __sub__(s, o): return s + (o * Fr(-1) if isinstance(o, EV) else -o)
    def __mul__(s, o):
        if isinstance(o, EV): return EV([a * b for a, b in zip(s.v, o.v)], [a * b for a, b in zip(s.m, o.m)])
        return EV([a * o for a in s.v], [m * abs(o) for m in s.m])
    __rmul__ = __mul__
    def __pow__(s, k):
        out = s
        for _ in range(k - 1): out = out * s
        return out


def apply_exact(Ls, e):
    """정확한 행렬 L*을 EV에 적용(크기는 |L*||m|)."""
    n = len(Ls)
    return EV([sum(Ls[i][j] * e.v[j] for j in range(n) if Ls[i][j] != 0) for i in range(n)],
              [sum(abs(Ls[i][j]) * e.m[j] for j in range(n) if Ls[i][j] != 0) for i in range(n)])


def check_formula(Fs, x, kind, formula, states, name="공식", level=None, k_ops=K_SP * 4):
    """아무 계수의 (비선형) 공간 공식 검사.  formula(Lx, u) -> EV: 정확한 그래프 연산자 Lx(EV -> EV)와 상태 u(EV)로 쓴 주장한 식
    (예: 칸–힐리어드 L(u^3 - u - kappa L u)).  상태마다 |Fs_i - F*_i| <= gam(k_ops) * 크기_i 를 검사하고, 상수 상태에서도 검사한다."""
    Ls = matrix_from_VT(*claim_graph(kind, x)); Lx = lambda e: apply_exact(Ls, e); g = gamF(k_ops); viol = []; n = len(Ls)
    for si, u in enumerate(list(states)):
        uf = np.asarray(u, float); Fv = np.asarray(Fs(uf.copy()), float); ue = EV.of([exact(v) for v in uf]); E = formula(Lx, ue)
        for i in range(n):
            d = abs(exact(Fv[i]) - E.v[i])
            if d > g * E.m[i]:
                viol.append(dict(level=level, state=si, i=i, j=None, rule=f"관계식({name})", stored=floa
```

```
    t(Fv[i]), expected=E.v[i], excess=float(d - g * E.m[i])))
368     return dict(ok=not viol, viol=viol, n=n, rules=f"{name}의 관계식", kind=f"{kind} 위의 {name}", gk=k_
    ops)
```

pinn_cert.py — PINN 출력과 방정식의 어긋남: 검은 상자를 상수 후보로, 위·아래 한계, 육하원칙과 다시 판정한 처방(17절, 임의 차원)

```
  1 """pinn_cert: PINN 출력과 방정식의 어긋남 — 검은 상자 신경망의 미지수를 상수로 명시하고, 적분과 부등식으
    로 증명하거나 반례를 낸다.
  2
  3 PINN은 가중치라는 미지수 덩어리이다.  검증기는 가중치·손실·구조에 접근하지 않고(어떤 PINN 라이브러리의 A
    PI도 쓰지
  4 않는다), 함수 u(x, t)를 정해 둔 표본점에서 부를 뿐이다.  표본점은 공간 그래프의 마디 x_i(4.4절)와 시간
    조각
  5 [t_k, t_k + h]마다의 K개 시각(체비쇼프-로바토 점)이다.  그 값으로 조각마다 시간 K-1차 다항식의 계수 V[a]
     (N, K)를
  6 정하면, PINN은 N * n * K개의 상수로 완전히 명시된 후보가 된다(경로 B의 조각, pde_judge.Piece).
  7     상수 후보:  u~(x, t) = 공간 3차 스플라인( sum_k V_ik (t - t_k)^k ),  표본점에서 u~ = PINN (차이 delt
    a는 계산해 적는다)
  8 판정(비선형 → 적분과 부등식, 경로 B):
  9     위:  ||u~(t_k) - u(t_k)||_1 <= E_k          (정리 12.1: 잔차의 적분, 가토 부등식, 연쇄곱; 가정 H)
 10     아래: ||u~(t_k) - u(t_k)||_1 >= ||u~(t_k) - u_ref(t_k)||_1(아래 한계) - beta_ref(t_k)
 11           u_ref는 세밀한 시공간 갤러킨 후보, beta_ref는 그 후보 자신의 경로 B 한도(인증된 기준해: e의 역
    할)
 12     에러 없음(증명): 모든 t_k에서 E_k <= eps.   정확도 에러(반례): 아래 한계 > eps인 첫 t_k.
 13 판정이 말하는 대상은 상수 후보 u~이다.  표본점 밖에서 신경망과 u~의 차이는 이 판정의 범위 밖이다(정의)."
    ""
 14 import math
 15 import numpy as np
 16 from pde_judge import Piece, Judge, localize, spline_slopes, cubic_coeffs
 17 from rig import gam
 18
 19
 20 def sample_times(h, K):
 21     return h * (1 - np.cos(np.pi * np.arange(K) / (K - 1))) / 2          # 체비쇼프-로바토 점 (0과 h 포
    함)
 22
 23
 24 def pieces_from_blackbox(u, x, T, n, K=5):
 25     """검은 상자 u(x, t) -> 조각들.  반환 (pieces, delta, 상수 개수)."""
 26     h = T / n; s = sample_times(h, K); Vm = s[:, None] ** np.arange(K)[None, :]; pieces = []; delta = 0.
    0
 27     for k in range(n):
 28         U = np.array([np.asarray(u(x, k * h + sq), float) for sq in s])        # (K, N)
 29         C = np.linalg.solve(Vm, U)                                              # (K, N): 시간 계수
 30         delta = max(delta, float(np.max(np.abs(Vm @ C - U))))
 31         pieces.append(Piece(k * h, h, x.copy(), [C.T]))
 32     return pieces, delta, len(x) * n * K
 33
 34
 35 def end_values(Pc):
 36     return Pc.V[0] @ (Pc.h ** np.arange(Pc.V[0].shape[1]))
 37
 38
 39 def spline_at(x, y):
 40     sl = spline_slopes(x, y[:, None], np.zeros(1), np.zeros(1)); return cubic_coeffs(x, y[:, None], sl)[
    :, :, 0]
 41
 42
 43 def l1_lower(xA, cA, xB, cB, M=4):
 44     """int |pA - pB|의 아래 한계(두 조각 3차 함수, 서로 다른 마디): 합집합 칸을 M개로 나누고 칸마다 |int
     f|
 45     (심프슨, 3차에 정확) 에서 계산의 반올림 상한을 뺀 것의 합."""
 46     xu = np.union1d(xA, xB); out = 0.0
 47     for a, b in zip(xu[:-1], xu[1:]):
 48         for q in range(M):
 49             lo, hi = a + (b - a) * q / M, a + (b - a) * (q + 1) / M; pts = np.array([lo, 0.5 * (lo + hi)
    , hi]); vals = np.zeros(3); mag = np.zeros(3)
 50             for X, Cc, sg in ((xA, cA, 1.0), (xB, cB, -1.0)):
 51                 j = np.clip(np.searchsorted(X, 0.5 * (lo + hi)) - 1, 0, len(X) - 2); ell = X[j + 1] - X[
    j]; xi = (pts - X[j]) / ell
 52                 vals += sg * (Cc[j][0] + xi * (Cc[j][1] + xi * (Cc[j][2] + xi * Cc[j][3]))); mag += np.a
    bs(Cc[j]).sum()
 53             S = (hi - lo) * (vals[0] + 4 * vals[1] + vals[2]) / 6
 54             out += max(abs(S) - (hi - lo) * (gam(16) * float(mag.max()) + gam(8) * float(np.abs(vals).ma
    x())), 0.0)
 55     return out * (1 - gam(4 * len(xu)))
 56
```

```
57
58  def aggregate(rows, T, nt=5, nxb=4, Lx=1.0):
59      """국소화 묶을 시공간 구역(시간 nt칸 × 공간 nxb칸)과 원인으로 모은다(묶은 그대로 더한다: 합은 정확).
    """
60      acc = {}
61      for r in rows:
62          if r["x"] is None: continue
63          it = min(int(r["t"][0] / T * nt), nt - 1); ix = min(int(0.5 * (r["x"][0] + r["x"][1]) / Lx * nxb
    ), nxb - 1)
64          key = (it, ix, r["cause"]); acc[key] = acc.get(key, 0.0) + r["share"]
65      out = [dict(t=[it * T / nt, (it + 1) * T / nt], x=[ix * Lx / nxb, (ix + 1) * Lx / nxb], cause=c, sha
    re=v, species=0) for (it, ix, c), v in acc.items()]
66      return sorted(out, key=lambda r: -r["share"])
67
68
69  def certify(u, spec, u0box, x, n, K, eps, ref, op=None, name="PINN"):
70      """ref = (ref_pieces, ref_series) — 세밀한 기준 후보와 그 경로 B 한도의 시계열(조각 끝마다).
71      반환: (보고 dict, 판정 결과 dict)."""
72      pieces, delta, ncon = pieces_from_blackbox(u, x, spec["T"], n, K)
73      J = Judge(spec); res = J.run(pieces, u0box, op=op, series_every=1); rows_all, tot = localize(res, to
    p=10 ** 6); rows = aggregate(rows_all, spec['T'])
74      up = {round(t, 12): e for t, e in res["series"]}
75      rpieces, rser = ref; rb = {round(t, 12): e for t, e in rser}
76      rend = {round(P.t0 + P.h, 12): P for P in rpieces}
77      lows, ups, ts = [], [], []
78      for P in pieces:
79          te = round(P.t0 + P.h, 12); ts.append(te); ups.append(up[te])
80          if te in rend and te in rb:
81              R = rend[te]; lows.append(max(l1_lower(P.x, spline_at(P.x, end_values(P)), R.x, spline_at(R.
    x, end_values(R))) - rb[te], 0.0))
82          else: lows.append(0.0)
83      bad = [i for i, l in enumerate(lows) if l > eps]; good = all(e <= eps for e in ups)
84      rep = {}
85      if good: rep["판정"] = dict(status="증명", text=f"에러 없음: 모든 조각 끝에서 L¹ 오차 ≤ {eps:g}(증명
    , 가정 H 아래)")
86      elif bad: rep["판정"] = dict(status="반례", text=f"정확도 에러: t = {ts[bad[0]]:.4g}에서 L¹ 오차 > {
    eps:g}(반례)")
87      else:
88          kk = int(np.argmax(np.array(ups) > eps))
89          rep["판정"] = dict(status="증명", text=f"허용치 경계: t = {ts[kk]:.4g}에서 L¹ 오차가 [{lows[kk]:
    .3e}, {ups[kk]:.3e}]이고 ε = {eps:g}를 품는다(증명된 해상도 문장)")
90      rep["누가"] = dict(status="정의", text=f"{name}(검은 상자: 가중치·손실·구조에 접근하지 않음). 상수로
     명시: 마디 {len(x)}개 × 시간 조각 {n}개 × 표본 시각 {K}개 = {ncon}개의 값과 "
91                         f"그 시간 다항식 계수; 표본점에서 상수 후보와 PINN의 차이 {delta:.1e}(계산)")
92      rep["무엇을"] = dict(status="증명", text=f"t = {ts[-1]:.4g}에서 L¹ 오차 ∈ [{lows[-1]:.3e}, {ups[-1]:
    .3e}] (위: 잔차의 적분과 부등식, 아래: 인증된 기준해와의 차이)")
93      k_ok = 0
94      while k_ok < len(ups) and ups[k_ok] <= eps: k_ok += 1
95      rep["언제"] = dict(status="증명" if not bad else "반례", text=(f"[0, {ts[k_ok - 1]:.4g}]에서 L¹ 오차
     ≤ {eps:g}(증명)" if k_ok else f"첫 조각부터 ≤ {eps:g}가 증명되지 않는다")
96                         + (f"; t = {ts[bad[0]]:.4g}에서 처음으로 > {eps:g}(반례)" if bad else ""))
97      cause_k = dict(initial="초기 오차", time_scheme="시간 방향 결함", space="공간 방향 결함", residual="
    잔차", jump="조각 사이 도약", rounding="반올림")
98      def loc(r):
99          xs = "조각 경계 전체" if r["x"] is None else f"x ∈ [{r['x'][0]:.3g}, {r['x'][1]:.3g}]"
100         return f"t ∈ [{r['t'][0]:.4g}, {r['t'][1]:.4g}], {xs}, {cause_k.get(r['cause'], r['cause'])} (묶
     {r['share']:.3f})"
101     rep["어디서"] = dict(status="정의(합은 증명)", text="; ".join(loc(r) for r in rows[:3]))
102     rep["왜"] = dict(status="정의(합은 증명)", text="적분 부등식의 원인별 묶: " + ", ".join(f"{cause_k.g
    et(k, k)} {v:.3f}" for k, v in sorted(tot.items(), key=lambda kv: -kv[1]) if v > 5e-4))
103     return rep, dict(good=good, bad=bad, rows=rows, tot=tot, ups=ups, lows=lows, ts=ts, res=res)
104
105
106 def certify_with_prescription(u, spec, u0box, x, n, K, eps, ref, op=None, name="PINN", retrain=None, max
    _steps=5):
107     """육하원칙 보고 + 처방(어떻게)을 실제로 적용해 다시 판정한 결과.  처방의 선택(지배 원인에 따라):
108       초기 오차·공간 방향 결함 → 표본 마디를 두 배로(검증기 쪽, PINN은 그대로)
109       시간 방향 결함 → 먼저 시간 조각을 두 배로; 그래도 같으면 PINN 자신의 결함이므로 retrain(구역)
110                        (대상 쪽: 국소화가 짚은 구역에 콜로케이션 점을 더해 이어서 학습)으로 고친 PINN을 다
    시 판정"""
111     rep, out = certify(u, spec, u0box, x, n, K, eps, ref, op, name); how = []; cur = (u, x, n)
112     for step in range(max_steps):
113         if out["good"]: break
114         dom = max(out["tot"], key=out["tot"].get); uu, xx, nn = cur
115         if dom in ("initial", "space", "rounding", "jump"):
116             xx = np.linspace(xx[0], xx[-1], 2 * len(xx) - 1); what = f"표본 마디를 {len(xx)}개로 늘림(PI
    NN은 그대로)"
117         elif nn < 4 * n:
118             nn = 2 * nn; what = f"시간 조각을 {nn}개로 늘림(PINN은 그대로)"
119         elif retrain is not None:
```

```
            r0 = out["rows"][0]; uu = retrain((r0["t"][0], r0["t"][1], r0["x"][0], r0["x"][1])); nn = n
            what = f"국소화가 짚은 구역 t ∈ [{r0['t'][0]:.3g}, {r0['t'][1]:.3g}], x ∈ [{r0['x'][0]:.3g}, {r0['x'][1]:.3g}]에 콜로케이션 점을 더해 이어서 학습(대상 쪽)"
        else: break
        cur = (uu, xx, nn); _, out = certify(uu, spec, u0box, xx, nn, K, eps, ref, op, name)
        how.append(f"{what} → " + ("모든 조각 끝에서 ≤ ε 증명" if out["good"] else ("반례 유지" if out["bad"] else "경계")))
    if how:
        rep["어떻게"] = dict(status="증명" if out["good"] else ("반례" if out["bad"] else "증명"),
                             text="처방을 적용해 다시 판정: " + "; ".join(how) + ("(처방 확인)" if out["good"] else ""))
    else:
        rep["어떻게"] = dict(status="증명", text="처방 필요 없음(에러 없음이 증명됨)")
    return rep, out


# ============================================================ 임의 차원(경로 B의 d차원 판, pde_judge_nd)
def pieces_from_blackbox_nd(u, xs, T, n, K=5):
    """검은 상자 u(X, t), X: (d, P) 점 묶음 -> d차원 조각.  마디 = 축별 마디의 텐서 격자, 시간 = 조각마다 로바토 점 K개."""
    from pde_judge_nd import PieceND
    h = T / n; s = sample_times(h, K); Vm = s[:, None] ** np.arange(K)[None, :]; shp = tuple(len(x) for x in xs)
    G = np.meshgrid(*xs, indexing="ij"); X = np.stack([g.ravel() for g in G]); pieces = []; delta = 0.0
    for k in range(n):
        U = np.array([np.asarray(u(X, k * h + sq), float) for sq in s])          # (K, P)
        C = np.linalg.solve(Vm, U); delta = max(delta, float(np.max(np.abs(Vm @ C - U))))
        pieces.append(PieceND(k * h, h, xs, [C.T.reshape(shp + (K,))]))
    return pieces, delta, int(np.prod(shp)) * n * K


def end_values_nd(P):
    return np.tensordot(P.V[0], P.h ** np.arange(P.V[0].shape[-1]), axes=([-1], [0]))


def l1_lower_nd(xsA, YA, xsB, YB):
    """int |pA - pB|의 아래 한계(두 텐서 3차 스플라인, 서로 다른 텐서 마디): 축마다 합집합 마디로 칸을 나누고,
    칸마다 |int f|(축별 심프슨의 텐서곱, 축마다 3차에 정확)에서 계산의 반올림 상한을 뺀 것의 합."""
    from pde_judge_nd import tensor_spline, eval_tensor
    d = len(xsA); cA = tensor_spline(xsA, YA[..., None])[..., 0]; cB = tensor_spline(xsB, YB[..., None])[..., 0]
    U = [np.union1d(a, b) for a, b in zip(xsA, xsB)]
    P = [np.sort(np.concatenate([u_, 0.5 * (u_[:-1] + u_[1:])])) for u_ in U]          # 마디와 가운데
    FA = eval_tensor(xsA, cA, P); FB = eval_tensor(xsB, cB, P); F = FA - FB; M = np.abs(FA) + np.abs(FB)
    w3 = np.array([1.0, 4.0, 1.0]) / 6; S = F; Mg = M
    for ax in range(d):                                                            # 축마다 칸 적분(심프슨)
        n_ = len(U[ax]) - 1; ell = np.diff(U[ax])
        sl = lambda A, o: np.take(A, np.arange(o, 2 * n_ + o, 2), axis=ax)
        S = (w3[0] * sl(S, 0) + w3[1] * sl(S, 1) + w3[2] * sl(S, 2)) * ell.reshape([-1 if i == ax else 1 for i in range(d)])
        Mg = np.maximum(np.maximum(sl(Mg, 0), sl(Mg, 1)), sl(Mg, 2))
    vol = np.ones_like(S)
    for ax in range(d): vol = vol * np.diff(U[ax]).reshape([-1 if i == ax else 1 for i in range(d)])
    marg = vol * (gam(16 * d + 16) * Mg * 4 ** d)
    return float(np.sum(np.maximum(np.abs(S) - marg, 0.0))) * (1 - gam(4 * S.size))


def aggregate_nd(rows, T, d, nt=5, nb=2):
    acc = {}
    for r in rows:
        if r["x"] is None: continue
        it = min(int(r["t"][0] / T * nt), nt - 1); ix = tuple(min(int(0.5 * (a + b) * nb), nb - 1) for a, b in r["x"])
        key = (it, ix, r["cause"]); acc[key] = acc.get(key, 0.0) + r["share"]
    out = [dict(t=[it * T / nt, (it + 1) * T / nt], x=[[i / nb, (i + 1) / nb] for i in ix], cause=c, share=v) for (it, ix, c), v in acc.items()]
    return sorted(out, key=lambda r: -r["share"])


def certify_nd(u, spec, u0box, u0grad, xs, n, K, eps, ref, op=None, name="PINN"):
    """d차원 판정.  ref = (기준 조각들, 기준 한도 시계열).  반환 (보고, 결과)."""
    from pde_judge_nd import JudgeND, localize as loc_nd
    d = len(xs); pieces, delta, ncon = pieces_from_blackbox_nd(u, xs, spec["T"], n, K)
    res = JudgeND(dict(spec, dim=d)).run(pieces, u0box, op=op, u0grad=u0grad); rows_all, tot = loc_nd(res, top=10 ** 6)
    rows = aggregate_nd(rows_all, spec["T"], d)
    up = {round(t, 12): e for t, e in res["series"]}; rp, rser = ref; rb = {round(t, 12): e for t, e in rser}
```